MINIBANK PROJECT

MiniBank Project

Account.java

```
public abstract class Account {

    private final String accountNumber;
    private String ownerName;
    private long balance;
    private boolean active;

    private static int accountCounter = 0;

    private static String generateAccountNumber() {
        accountCounter++;
        return String.format("AC%04d", accountCounter);
    }

    public Account(String ownerName, long balance) {
        this.accountNumber = generateAccountNumber();
        this.ownerName = ownerName;
        this.balance = balance;
        this.active = true;
    }

    public Account(String ownerName) {
        this(ownerName, 0);
    }

    public abstract double interestRate();

    public abstract boolean canWithdraw(long amount);

    public synchronized void deposit(long amount) {
        if (amount > 0) {
            balance += amount;
        }
    }

    public synchronized boolean withdraw(long amount) {
        if (canWithdraw(amount)) {
            balance -= amount;
            return true;
        }
        return false;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public String getOwnerName() {
        return ownerName;
    }
```

MINIBANK PROJECT

```
    public synchronized long getBalance() {
        return balance;
    }

    public boolean isActive() {
        return active;
    }

    public void setActive(boolean active) {
        this.active = active;
    }

    @Override
    public String toString() {
        return accountNumber + " | " + ownerName + " | " + getBalance();
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Account)) return false;

        Account other = (Account) o;
        return accountNumber.equals(other.accountNumber);
    }

    @Override
    public int hashCode() {
        return accountNumber.hashCode();
    }
}
```

MINIBANK PROJECT

Customer.java

```
public class Customer implements Cloneable {

    private static int customerCounter = 100;

    private final String customerId;
    private String name;
    private String email;
    private String mobile;
    private Address address;

    public Customer(String name, String email, String mobile) {
        this.customerId = "CUST" + (++customerCounter);
        this.name = name;
        this.email = email;
        this.mobile = mobile;
    }

    public String getCustomerId() {
        return customerId;
    }

    public String getName() {
        return name;
    }

    public String getEmail() {
        return email;
    }

    public String getMobile() {
        return mobile;
    }

    public Address getAddress() {
        return address;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public void setMobile(String mobile) {
        this.mobile = mobile;
    }

    public void setAddress(Address address) {
        this.address = address;
    }

    public static class Address {
        private String line;
        private String city;
        private String pincode;

        public Address(String line, String city, String pincode) {
```

MINIBANK PROJECT

```
            this.line = line;
            this.city = city;
            this.pincode = pincode;
        }

        public String getLine() {
            return line;
        }

        public String getCity() {
            return city;
        }

        public String getPincode() {
            return pincode;
        }
    }

    @Override
    public Customer clone() {
        try {
            Customer copy = (Customer) super.clone();

            if (address != null) {
                copy.address = new Address(
                    address.getLine(),
                    address.getCity(),
                    address.getPincode()
                );
            }

            return copy;
        } catch (CloneNotSupportedException e) {
            throw new AssertionError(e);
        }
    }
}
```

MINIBANK PROJECT

SavingsAccount.java

```
public class SavingsAccount extends Account {

    private long minBalance;

    public SavingsAccount(String ownerName, long balance, long minBalance) {
        super(ownerName, balance);
        this.minBalance = minBalance;
    }

    @Override
    public double interestRate() {
        return 4.0;
    }

    @Override
    public boolean canWithdraw(long amount) {
        return amount > 0 && getBalance() - amount >= minBalance;
    }

    public long getMinBalance() {
        return minBalance;
    }
}
```

MINIBANK PROJECT

CurrentAccount.java

```
public class CurrentAccount extends Account {

    private long overdraftLimit;

    public CurrentAccount(String ownerName, long balance, long overdraftLimit) {
        super(ownerName, balance);
        this.overdraftLimit = overdraftLimit;
    }

    @Override
    public double interestRate() {
        return 0.0;
    }

    @Override
    public boolean canWithdraw(long amount) {
        return amount > 0 &&
               getBalance() - amount >= -overdraftLimit;
    }

    public long getOverdraftLimit() {
        return overdraftLimit;
    }
}
```

MINIBANK PROJECT

FixedDepositAccount.java

```
public class FixedDepositAccount extends Account {

    public FixedDepositAccount(String ownerName, long balance) {
        super(ownerName, balance);
    }

    @Override
    public double interestRate() {
        return 7.0;
    }

    @Override
    public boolean canWithdraw(long amount) {
        return false;
    }
}
```

MINIBANK PROJECT

TransactionType.java

```
public enum TransactionType {
    DEPOSIT,
    WITHDRAW,
    TRANSFER
}
```

MINIBANK PROJECT

Command.java

```
public record Command(
    TransactionType type,
    String accountNumber,
    long amount
) {
}
```

MINIBANK PROJECT

CommandParser.java

```
public class CommandParser {

    public static Command parse(String line) {
        if (line == null || line.trim().isEmpty()) {
            return null;
        }

        String[] parts = line.trim().split("\\s+");

        if (parts.length != 3) {
            return null;
        }

        try {
            TransactionType type =
                TransactionType.valueOf(parts[0].toUpperCase());

            String accountNumber = parts[1];
            long amount = Long.parseLong(parts[2]);

            if (amount <= 0) {
                return null;
            }

            return new Command(type, accountNumber, amount);
        } catch (IllegalArgumentException e) {
            return null;
        }
    }
}
```

MINIBANK PROJECT

StatementFormatter.java

```
public class StatementFormatter {

    public static String buildStatement(Account account) {
        StringBuilder statement = new StringBuilder();

        statement.append("----- Account Statement -----\n");
        statement.append("Account Number: ")
                 .append(account.getAccountNumber()).append("\n");
        statement.append("Owner Name: ")
                 .append(account.getOwnerName()).append("\n");
        statement.append("Balance: ")
                 .append(account.getBalance()).append("\n");
        statement.append("Interest Rate: ")
                 .append(account.interestRate()).append("%\n");
        statement.append("Active: ")
                 .append(account.isActive()).append("\n");
        statement.append("----------------------------");

        return statement.toString();
    }
}
```

MINIBANK PROJECT

Validator.java

```
import java.util.regex.Pattern;

public class Validator {

    private static final Pattern MOBILE =
        Pattern.compile("^[6-9][0-9]{9}$");

    private static final Pattern EMAIL =
        Pattern.compile("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$");

    private static final Pattern PAN =
        Pattern.compile("^[A-Z]{5}[0-9]{4}[A-Z]$");

    private static final Pattern IFSC =
        Pattern.compile("^[A-Z]{4}0[A-Z0-9]{6}$");

    public static boolean isValidMobile(String mobile) {
        return mobile != null && MOBILE.matcher(mobile).matches();
    }

    public static boolean isValidEmail(String email) {
        return email != null && EMAIL.matcher(email).matches();
    }

    public static boolean isValidPan(String pan) {
        return pan != null && PAN.matcher(pan).matches();
    }

    public static boolean isValidIfsc(String ifsc) {
        return ifsc != null && IFSC.matcher(ifsc).matches();
    }
}
```

MINIBANK PROJECT

Transactable.java

```
public interface Transactable {
    void deposit(long amount);
    boolean withdraw(long amount);
}
```

MINIBANK PROJECT

InterestBearing.java

```
public interface InterestBearing {

    double interestRate();

    default double yearlyInterest(long balance) {
        return balance * interestRate() / 100.0;
    }
}
```

MINIBANK PROJECT

WithdrawRule.java

```
@FunctionalInterface
public interface WithdrawRule {
    boolean canWithdraw(long balance, long amount);
}
```

MINIBANK PROJECT

Premium.java

```
public interface Premium {
}
```

MINIBANK PROJECT

Id.java

```
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.annotation.ElementType;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface Id {
}
```

MINIBANK PROJECT

Positive.java

```
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.annotation.ElementType;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface Positive {
    String message() default "must be > 0";
}
```

MINIBANK PROJECT

MaxLength.java

```
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.annotation.ElementType;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface MaxLength {
    int value();
}
```

MINIBANK PROJECT

AnnotationValidator.java

```
import java.lang.reflect.Field;
import java.util.ArrayList;
import java.util.List;

public class AnnotationValidator {

    public static List<String> validate(Object object) {
        List<String> errors = new ArrayList<>();

        if (object == null) {
            errors.add("Object cannot be null");
            return errors;
        }

        Class<?> type = object.getClass();

        while (type != null) {
            for (Field field : type.getDeclaredFields()) {
                try {
                    field.setAccessible(true);
                    Object value = field.get(object);

                    if (field.isAnnotationPresent(Positive.class)) {
                        Positive annotation =
                            field.getAnnotation(Positive.class);

                        if (!(value instanceof Number) ||
                            ((Number) value).doubleValue() <= 0) {
                            errors.add(field.getName() + " " +
                                       annotation.message());
                        }
                    }

                    if (field.isAnnotationPresent(MaxLength.class)) {
                        MaxLength annotation =
                            field.getAnnotation(MaxLength.class);

                        if (value instanceof String &&
                            ((String) value).length() > annotation.value()) {
                            errors.add(field.getName() +
                                       " exceeds maximum length " +
                                       annotation.value());
                        }
                    }
                } catch (IllegalAccessException e) {
                    errors.add("Cannot access field: " + field.getName());
                }
            }

            type = type.getSuperclass();
        }

        return errors;
    }
}
```

MINIBANK PROJECT

BankException.java

```
public class BankException extends Exception {

    public BankException(String message) {
        super(message);
    }
}
```

MINIBANK PROJECT

InsufficientFundsException.java

```
public class InsufficientFundsException extends BankException {

    private final long shortfall;

    public InsufficientFundsException(long shortfall) {
        super("Insufficient funds. Shortfall: " + shortfall);
        this.shortfall = shortfall;
    }

    public long getShortfall() {
        return shortfall;
    }
}
```

MINIBANK PROJECT

AccountNotFoundException.java

```
public class AccountNotFoundException extends BankException {

    public AccountNotFoundException(String accountNumber) {
        super("Account not found: " + accountNumber);
    }
}
```

MINIBANK PROJECT

InvalidAmountException.java

```
public class InvalidAmountException extends BankException {

    public InvalidAmountException(long amount) {
        super("Invalid transaction amount: " + amount);
    }
}
```

MINIBANK PROJECT

BankService.java

```
public class BankService {

    public void deposit(Account account, long amount)
            throws InvalidAmountException {
        if (amount <= 0) {
            throw new InvalidAmountException(amount);
        }

        account.deposit(amount);
    }

    public void withdraw(Account account, long amount)
            throws BankException {
        if (amount <= 0) {
            throw new InvalidAmountException(amount);
        }

        long balance = account.getBalance();

        if (!account.canWithdraw(amount)) {
            long shortfall = amount - balance;
            if (shortfall < 1) {
                shortfall = 1;
            }
            throw new InsufficientFundsException(shortfall);
        }

        account.withdraw(amount);
    }

    public void transfer(Account from, Account to, long amount)
            throws BankException {
        if (amount <= 0) {
            throw new InvalidAmountException(amount);
        }

        try {
            withdraw(from, amount);
            to.deposit(amount);
        } catch (BankException e) {
            throw e;
        } finally {
            System.out.println("Transfer operation completed.");
        }
    }

    public Account findAccount(Account[] accounts, String accountNumber)
            throws AccountNotFoundException {
        for (Account account : accounts) {
            if (account != null &&
                account.getAccountNumber().equals(accountNumber)) {
                return account;
            }
        }

        throw new AccountNotFoundException(accountNumber);
    }
}
```

MINIBANK PROJECT

MINIBANK PROJECT

AccountWorker.java

```
public class AccountWorker implements Runnable {

    private final Account account;
    private final int repetitions;
    private final long amount;

    public AccountWorker(Account account, int repetitions, long amount) {
        this.account = account;
        this.repetitions = repetitions;
        this.amount = amount;
    }

    @Override
    public void run() {
        for (int i = 0; i < repetitions; i++) {
            account.deposit(amount);
        }
    }

    public static void main(String[] args) throws InterruptedException {
        Account account = new CurrentAccount("Test", 0, 1000000);

        Thread t1 = new Thread(new AccountWorker(account, 1000, 10));
        Thread t2 = new Thread(new AccountWorker(account, 1000, 10));
        Thread t3 = new Thread(new AccountWorker(account, 1000, 10));

        t1.start();
        t2.start();
        t3.start();

        t1.join();
        t2.join();
        t3.join();

        System.out.println("Final balance: " + account.getBalance());
    }
}
```

MINIBANK PROJECT

TransactionProcessor.java

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class TransactionProcessor implements AutoCloseable {

    private final ExecutorService executor =
        Executors.newFixedThreadPool(4);

    public void submit(Runnable transaction) {
        executor.submit(transaction);
    }

    public void stop() {
        executor.shutdown();
    }

    public void awaitCompletion() throws InterruptedException {
        if (!executor.awaitTermination(10, TimeUnit.SECONDS)) {
            executor.shutdownNow();
        }
    }

    @Override
    public void close() {
        stop();

        try {
            awaitCompletion();
        } catch (InterruptedException e) {
            executor.shutdownNow();
            Thread.currentThread().interrupt();
        }
    }

    public static void main(String[] args) {
        Account account = new CurrentAccount("Kanan", 0, 1000000);

        try (TransactionProcessor processor = new TransactionProcessor()) {
            for (int i = 0; i < 100; i++) {
                processor.submit(() -> account.deposit(100));
            }
        }

        System.out.println("Final balance: " + account.getBalance());
    }
}
```

MINIBANK PROJECT

ProducerConsumer.java

```
import java.util.LinkedList;
import java.util.Queue;

public class ProducerConsumer {

    private final Queue<Integer> queue = new LinkedList<>();
    private final int capacity = 5;

    public synchronized void produce(int value)
            throws InterruptedException {
        while (queue.size() == capacity) {
            wait();
        }

        queue.add(value);
        System.out.println("Produced: " + value);
        notifyAll();
    }

    public synchronized int consume() throws InterruptedException {
        while (queue.isEmpty()) {
            wait();
        }

        int value = queue.remove();
        System.out.println("Consumed: " + value);
        notifyAll();

        return value;
    }

    public static void main(String[] args) {
        ProducerConsumer pc = new ProducerConsumer();

        Thread producer = new Thread(() -> {
            try {
                for (int i = 1; i <= 10; i++) {
                    pc.produce(i);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        Thread consumer = new Thread(() -> {
            try {
                for (int i = 1; i <= 10; i++) {
                    pc.consume();
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        producer.start();
        consumer.start();
    }
}
```

MINIBANK PROJECT

MINIBANK PROJECT

DeadlockDemo.java

```
public class DeadlockDemo {

    public static void transfer(Account first, Account second, long amount) {
        Account lower = first.getAccountNumber()
                .compareTo(second.getAccountNumber()) < 0 ? first : second;

        Account higher = lower == first ? second : first;

        synchronized (lower) {
            synchronized (higher) {
                if (first.canWithdraw(amount)) {
                    first.withdraw(amount);
                    second.deposit(amount);
                }
            }
        }
    }

    public static void main(String[] args) throws InterruptedException {
        Account a = new CurrentAccount("A", 1000, 1000);
        Account b = new CurrentAccount("B", 1000, 1000);

        Thread t1 = new Thread(() -> transfer(a, b, 100));
        Thread t2 = new Thread(() -> transfer(b, a, 100));

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println(a);
        System.out.println(b);
    }
}
```

MINIBANK PROJECT

MiniBank.java

```
import java.util.Scanner;

public class MiniBank {

    record BankInfo(String name, String branch) {}

    enum MenuOption {
        OPEN_ACCOUNT, DEPOSIT, WITHDRAW, TRANSFER, EXIT
    }

    private static Account[] accounts = new Account[100];
    private static int accountCount = 0;

    private static Account findAccount(String accountNumber) {
        for (int i = 0; i < accountCount; i++) {
            if (accounts[i].getAccountNumber().equals(accountNumber)) {
                return accounts[i];
            }
        }
        return null;
    }

    private static void showAccounts() {
        if (accountCount == 0) {
            System.out.println("No accounts available.");
            return;
        }

        for (int i = 0; i < accountCount; i++) {
            System.out.println(accounts[i]);
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        BankInfo bank = new BankInfo("MiniBank", "CHARUSAT");

        System.out.println("-----------------------------");
        System.out.println(bank.name() + " - " + bank.branch());
        System.out.println("-----------------------------");

        while (true) {
            System.out.println("\\n1. Open account");
            System.out.println("2. Deposit");
            System.out.println("3. Withdraw");
            System.out.println("4. Transfer");
            System.out.println("5. View accounts");
            System.out.println("6. Exit");
            System.out.print("Enter your choice: ");

            if (!sc.hasNextInt()) {
                System.out.println("Enter a valid number.");
                sc.nextLine();
                continue;
            }

            int choice = sc.nextInt();
            sc.nextLine();
```

MINIBANK PROJECT

```
MenuOption option;

switch (choice) {
    case 1 -> option = MenuOption.OPEN_ACCOUNT;
    case 2 -> option = MenuOption.DEPOSIT;
    case 3 -> option = MenuOption.WITHDRAW;
    case 4 -> option = MenuOption.TRANSFER;
    case 6 -> option = MenuOption.EXIT;
    case 5 -> {
        showAccounts();
        continue;
    }
    default -> {
        System.out.println("Invalid choice.");
        continue;
    }
}

switch (option) {
    case OPEN_ACCOUNT -> {
        if (accountCount >= accounts.length) {
            System.out.println("Account limit reached.");
            break;
        }

        System.out.print("Enter owner name: ");
        String name = sc.nextLine();

        System.out.println("1. Savings  2. Current  3. Fixed Deposit");
        System.out.print("Choose account type: ");

        if (!sc.hasNextInt()) {
            System.out.println("Invalid account type.");
            sc.nextLine();
            break;
        }

        int type = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter initial balance: ");
        if (!sc.hasNextLong()) {
            System.out.println("Invalid balance.");
            sc.nextLine();
            break;
        }

        long balance = sc.nextLong();
        sc.nextLine();

        if (balance < 0) {
            System.out.println("Balance cannot be negative.");
            break;
        }

        Account account;

        if (type == 1) {
            System.out.print("Enter minimum balance: ");
            if (!sc.hasNextLong()) {
```

MINIBANK PROJECT

```
                System.out.println("Invalid minimum balance.");
                sc.nextLine();
                break;
            }
            long minimum = sc.nextLong();
            sc.nextLine();
            account = new SavingsAccount(name, balance, minimum);
        } else if (type == 2) {
            System.out.print("Enter overdraft limit: ");
            if (!sc.hasNextLong()) {
                System.out.println("Invalid overdraft limit.");
                sc.nextLine();
                break;
            }
            long limit = sc.nextLong();
            sc.nextLine();
            account = new CurrentAccount(name, balance, limit);
        } else if (type == 3) {
            account = new FixedDepositAccount(name, balance);
        } else {
            System.out.println("Invalid account type.");
            break;
        }

        accounts[accountCount++] = account;
        System.out.println("Account created successfully.");
        System.out.println(account);
    }

    case DEPOSIT -> {
        System.out.print("Enter account number: ");
        String number = sc.nextLine();
        Account account = findAccount(number);

        if (account == null) {
            System.out.println("Account not found.");
            break;
        }

        System.out.print("Enter amount: ");
        if (!sc.hasNextLong()) {
            System.out.println("Invalid amount.");
            sc.nextLine();
            break;
        }

        long amount = sc.nextLong();
        sc.nextLine();

        if (amount <= 0) {
            System.out.println("Amount must be positive.");
        } else {
            account.deposit(amount);
            System.out.println("Deposit successful.");
            System.out.println(account);
        }
    }

    case WITHDRAW -> {
        System.out.print("Enter account number: ");
```

MINIBANK PROJECT

```
    String number = sc.nextLine();
    Account account = findAccount(number);

    if (account == null) {
        System.out.println("Account not found.");
        break;
    }

    System.out.print("Enter amount: ");
    if (!sc.hasNextLong()) {
        System.out.println("Invalid amount.");
        sc.nextLine();
        break;
    }

    long amount = sc.nextLong();
    sc.nextLine();

    if (account.withdraw(amount)) {
        System.out.println("Withdrawal successful.");
    } else {
        System.out.println("Withdrawal denied.");
    }
    System.out.println(account);
}

case TRANSFER -> {
    System.out.print("Enter source account number: ");
    Account from = findAccount(sc.nextLine());

    System.out.print("Enter destination account number: ");
    Account to = findAccount(sc.nextLine());

    if (from == null || to == null) {
        System.out.println("Account not found.");
        break;
    }

    System.out.print("Enter amount: ");
    if (!sc.hasNextLong()) {
        System.out.println("Invalid amount.");
        sc.nextLine();
        break;
    }

    long amount = sc.nextLong();
    sc.nextLine();

    if (from == to || amount <= 0) {
        System.out.println("Invalid transfer.");
    } else if (from.withdraw(amount)) {
        to.deposit(amount);
        System.out.println("Transfer successful.");
    } else {
        System.out.println("Transfer denied.");
    }
}

case EXIT -> {
    System.out.println("Thank you for using MiniBank.");
```

MINIBANK PROJECT

```
                    sc.close();
                    return;
                }
            }
        }
    }
}
```

MINIBANK PROJECT

ScreenShot Output:

Main class:

```
------------------------------
MiniBank - CHARUSAT
------------------------------

1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. View accounts
6. Exit
Enter your choice:
```

Creating Bank Account:

```
Enter your choice: 1
Enter owner name: Kanan
1. Savings  2. Current  3. Fixed Deposit
Choose account type: 1
Enter initial balance: 1000
Enter minimum balance: 900
Account created successfully.
AC0001 | Kanan | 1000
```

MINIBANK PROJECT

Deposit :

```
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. View accounts
6. Exit
Enter your choice: 2
Enter account number: AC0001
Enter amount: 400
Deposit successful.
AC0001 | Kanan | 1400
```

Withdrawal:

```
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. View accounts
6. Exit
Enter your choice: 3
Enter account number: AC0001
Enter amount: 500
Withdrawal successful.
AC0001 | Kanan | 900
```

Transfer:

Transfer denied because if AC0001 transfers then it will go below minimum amount.

MINIBANK PROJECT

```
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. View accounts
6. Exit
Enter your choice: 4
Enter source account number: AC0001
Enter destination account number: AC0002
Enter amount: 420
Transfer denied.
```

Success.

```
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. View accounts
6. Exit
Enter your choice: 4
Enter source account number: AC0001
Enter destination account number: AC0002
Enter amount: 1000
Transfer successful.
```

View Accounts:

```
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. View accounts
6. Exit
Enter your choice: 5
AC0001 | Kanan | 2000
AC0002 | Someone | 4000
```